



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №3

із дисципліни «Технології розроблення програмного забезпечення»

Тема: «Вступ до паттернів проектування»

Тема проекту: «Музичний програвач»

GitHub: <https://github.com/martyshenkoo/MusicPlayerWeb>

Виконала:

студентка групи ІА-31:

Мартишенко І.С.

Перевірив:

Мягкий М.Ю.

Київ- 2025

Тема: Вступ до паттернів проектування

Мета: Вивчити структуру шаблонів «Singleton», «Iterator», «Proxy», «State», «Strategy» та навчитися застосовувати їх в реалізації програмної системи.

Тема проекту:

Музичний програвач (iterator, command, memento, facade, visitor, client-server)

Музичний програвач становить собою програму для програвання музичних файлів або відтворення потокової музики з можливістю створення, запам'ятовування і редагування списків програвання, перемішування/повторення (shuffle/repeat), розпізнавання різних аудіо-форматів, еквалайзер.

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Хід роботи

У межах проєкту «Музичний програвач» було реалізовано основну логіку роботи з користувачами та аудіотреками.

Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.

```
namespace MusicPlayerWeb.Models;

27 references
public class Track
{
    4 references
    public Guid Id { get; set; } = Guid.NewGuid();
    7 references
    public string Title { get; set; } = default!;
    2 references
    public string FileName { get; set; } = default!;
    3 references
    public string RelativeUrl { get; set; } = default!;
    2 references
    public DateTime AddedAt { get; set; } = DateTime.UtcNow;
    4 references
    public string? OwnerUsername { get; set; }
}
```

Рис-1. Models-Track.cs

```
Models > Playlist.cs > ...
1 namespace MusicPlayerWeb.Models;
2
15 references
3 public class Playlist
4 {
    11 references
5     public Guid Id { get; set; } = Guid.NewGuid();
    7 references
6     public string Title { get; set; } = default!;
    9 references
7     public string OwnerUsername { get; set; } = default!;
    1 reference
8     public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
9
    2 references
10    public ICollection<PlaylistTrack> PlaylistTracks { get; set; } = new List<PlaylistTrack>();
11
    1 reference
12    public ITrackIterator CreateIterator(IEnumerable<Track> tracks)
13    {
14        return new PlaylistIterator(tracks);
15    }
16 }
```

Рис-2. Models-Playlist.cs

lels > C# PlaylistTrack.cs > ...

```
namespace MusicPlayerWeb.Models;
```

6 references

```
public class PlaylistTrack  
{
```

1 reference

```
    public Guid Id { get; set; } = Guid.NewGuid();
```

7 references

```
    public Guid PlaylistId { get; set; }
```

5 references

```
    public Playlist Playlist { get; set; } = default!;
```

5 references

```
    public Guid TrackId { get; set; }
```

4 references

```
    public Track Track { get; set; } = default!;
```

3 references

```
    public int Order { get; set; }
```

```
} 
```

Рис-3. Models- PlaylistTrack.cs

services > C# DbPlaylistService.cs > DbPlaylistService > AddTrack

```
1  using Microsoft.EntityFrameworkCore;
2  using MusicPlayerWeb.Data;
3  using MusicPlayerWeb.Models;
4
5  namespace MusicPlayerWeb.Services;
6
7  2 references
8  public class DbPlaylistService : IPlaylistService
9  {
10     19 references
11     private readonly AppDbContext _db;
12
13     0 references
14     public DbPlaylistService(AppDbContext db) => _db = db;
15
16     2 references
17     public IEnumerable<Playlist> GetForUser(string username)
18     {
19         return _db.Playlists
20             .Where(p => p.OwnerUsername == username)
21             .OrderBy(p => p.CreatedAt)
22             .ToList();
23
24     3 references
25     public Playlist? GetById(string username, Guid id)
26     {
27         return _db.Playlists
28             .Include(p => p.PlaylistTracks)
29             .ThenInclude(pt => pt.Track)
30             .FirstOrDefault(p => p.Id == id && p.OwnerUsername == username);
31     }
32
33     2 references
```

2 references

```

public Playlist Create(string username, string title)
{
    var pl = new Playlist
    {
        Title = title.Trim(),
        OwnerUsername = username
    };

    _db.Playlists.Add(pl);
    _db.SaveChanges();
    return pl;
}

```

3 references

```

public IEnumerable<Track> GetTracks(string username, Guid playlistId)
{
    return _db.PlaylistTracks
        .Include(pt => pt.Track)
        .Include(pt => pt.Playlist)
        .Where(pt => pt.PlaylistId == playlistId &&
            pt.Playlist.OwnerUsername == username)
        .OrderBy(pt => pt.Order)
        .Select(pt => pt.Track)
        .ToList();
}

```

2 references

```

public bool AddTrack(string username, Guid playlistId, Guid trackId)
{
    var playlist = _db.Playlists.FirstOrDefault(p => p.Id == playlistId && p.OwnerUsername == username);
    if (playlist == null) return false;

    var exists = _db.PlaylistTracks.Any(pt => pt.PlaylistId == playlistId && pt.TrackId == trackId);
    if (exists) return true;

    int maxOrder = _db.PlaylistTracks
        .Where(pt => pt.PlaylistId == playlistId)
        .Select(pt => (int?)pt.Order)
        .Max() ?? 0;

    _db.PlaylistTracks.Add(new PlaylistTrack
    {
        PlaylistId = playlistId,
        TrackId = trackId,
        Order = maxOrder + 1
    });

    _db.SaveChanges();
    return true;
}

```

1 reference

```

public bool RemoveTrack(string username, Guid playlistId, Guid trackId)
{
    var pt = _db.PlaylistTracks
        .Include(x => x.Playlist)
        .FirstOrDefault(x =>
            x.PlaylistId == playlistId &&
            x.TrackId == trackId &&
            x.Playlist.OwnerUsername == username);

    if (pt == null) return false;

    _db.PlaylistTracks.Remove(pt);
    _db.SaveChanges();
    return true;
}

```

```

2 references
public bool Rename(string username, Guid playlistId, string newTitle)
{
    var pl = _db.Playlists.FirstOrDefault(p => p.Id == playlistId && p.OwnerUsername == username);
    if (pl == null) return false;

    pl.Title = newTitle.Trim();
    _db.SaveChanges();
    return true;
}

2 references
public bool Delete(string username, Guid playlistId)
{
    var pl = _db.Playlists.FirstOrDefault(p => p.Id == playlistId && p.OwnerUsername == username);
    if (pl == null) return false;

    _db.Playlists.Remove(pl);
    _db.SaveChanges();
    return true;
}
}

```

Рис- 4-7. Services- DbPlaylistService.cs

```

1 using Microsoft.AspNetCore.Authorization;
2 using Microsoft.AspNetCore.Mvc;
3 using MusicPlayerWeb.Models;
4 using MusicPlayerWeb.Services;
5
6 namespace MusicPlayerWeb.Controllers;
7
8 [Authorize]
9 public class HomeController : Controller
10 {
11     private readonly ITrackService _tracks;
12     private readonly IPlaylistService _playlists;
13
14     public HomeController(ITrackService tracks, IPlaylistService playlists)
15     {
16         _tracks = tracks;
17         _playlists = playlists;
18     }
19
20     public IActionResult Index(Guid? playlistId = null)
21     {
22         var username = User.Identity!.Name!;
23         var playlists = _playlists.GetForUser(username).ToList();
24         if (!playlistId.HasValue && playlists.Any())
25             playlistId = playlists.First().Id;
26
27         Playlist? selected = null;
28         IEnumerable<Track> tracks = Enumerable.Empty<Track>();
29
30         if (playlistId.HasValue)
31         {
32             selected = _playlists.GetById(username, playlistId.Value);
33             if (selected != null)
34                 tracks = _playlists.GetTracks(username, playlistId.Value);
35         }
36
37         var model = new PlaylistPageViewModel
38         {
39             Playlists = playlists,
40             SelectedPlaylistId = playlistId,
41             SelectedPlaylistTitle = selected?.Title,
42             Tracks = tracks
43         };
44
45         return View(model);
46     }
47 }

```

Рис-8. Controllers- HomeController.cs

Реалізувати один з розглянутих шаблонів за обраною темою.

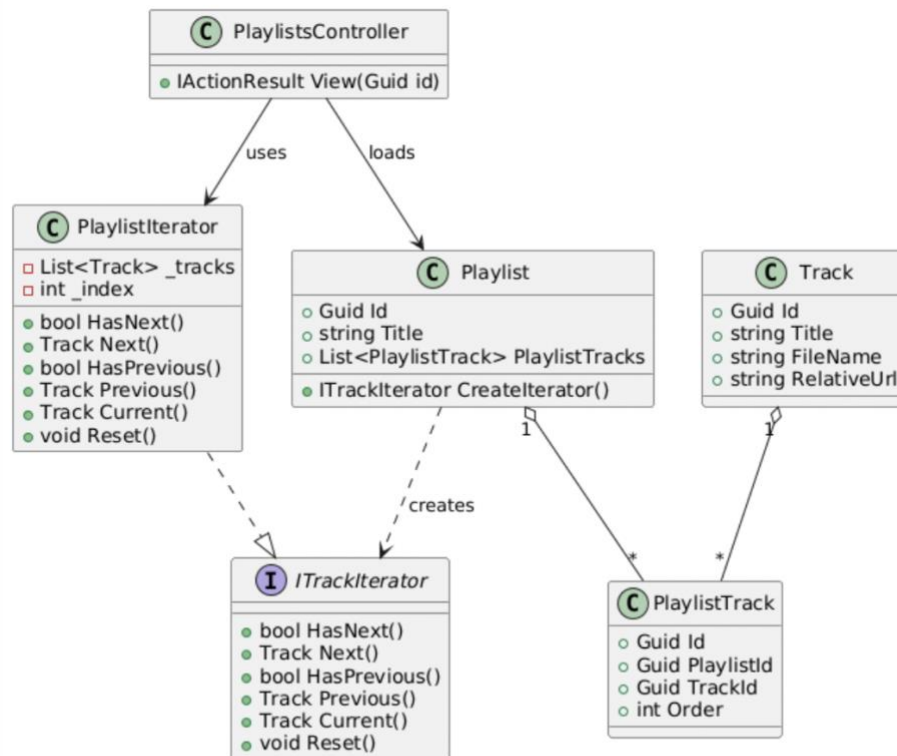


Рис-9. Структура патерну Ітератор

Для реалізації механізму послідовного відтворення треків у плейлісті було використано патерн Ітератор. Його структура в проєкті показана на діаграмі класів.

Інтерфейс ITrackIterator

Визначає спільний інтерфейс для всіх ітераторів. Містить методи для переміщення між треками:

- `HasNext()` - чи існує наступний трек
- `Next()` - перейти до наступного
- `HasPrevious()` - чи існує попередній

- Previous() - перейти до попереднього
- Current() - отримати поточний трек
- Reset() - повернення на початок

Це - основа патерну, яка забезпечує єдиний спосіб навігації.

Клас PlaylistIterator

Це конкретна реалізація ітератора, яка знає, як обходити треки плейліста. Він містить:

- внутрішній список треків List<Track> _tracks
- поточну позицію _index

І відповідає за фактичну логіку “перемотування” треків. Саме тут реалізовано весь механізм переходів вперед/назад.

Клас Playlist

Це музичний плейліст, який містить список треків у вигляді зв’язків

PlaylistTracks. Таким чином плейліст не показує свою внутрішню структуру, а лише повертає зручний спосіб обходу елементів.

Клас PlaylistTrack

Містить інформацію про трек у конкретному плейлісті та його позицію (Order).

Ця сутність використовується для формування впорядкованого списку треків, який використовує ітератор.

Клас PlaylistsController

Контролер є клієнтом патерну Iterator. У методі перегляду плейліста він:

- завантажує плейліст із БД
- викликає CreateIterator()
- отримує перший трек
- передає назви треків у View для навігації кнопками "Далі" та "Назад"

Таким чином саме контролер використовує ітератор, не отримуючи прямого доступу до структури колекції.

```
Models > C# ITrackIterator.cs > ...
1  namespace MusicPlayerWeb.Models;
2
   2 references
3  public interface ITrackIterator
4  {
   2 references
5      bool HasNext();
   1 reference
6      Track Next();
   2 references
7      bool HasPrevious();
   1 reference
8      Track Previous();
   4 references
9      Track Current();
10 }
11 |
```

Рис-10. Models-ITrackIterator.cs

```

deis > 📄 PlaylistIterator.cs > 📁 PlaylistIterator > 📄 HasNext
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4
5  namespace MusicPlayerWeb.Models;
6
7  2 references
8  public class PlaylistIterator : ITrackIterator
9  {
10     5 references
11     private readonly IList<Track> _tracks;
12     9 references
13     private int _index;
14
15     1 reference
16     public PlaylistIterator(IEnumerable<Track> tracks)
17     {
18         _tracks = tracks.ToList();
19         _index = _tracks.Count > 0 ? 0 : -1;
20     }
21
22     2 references
23     public bool HasNext() => _index >= 0 && _index < _tracks.Count - 1;
24
25     1 reference
26     public Track Next()
27     {
28         if (HasNext())
29         {
30             _index++;
31         }
32         return Current();
33     }
34
35     2 references
36     public bool HasPrevious() => _index > 0;
37
38     1 reference
39     public Track Previous()
40     {
41         if (HasPrevious())
42         {
43             _index--;
44         }
45         return Current();
46     }
47
48     4 references
49     public Track Current()
50     {
51         if (_index < 0 || _index >= _tracks.Count)
52             throw new InvalidOperationException("Немає поточного треку.");
53         return _tracks[_index];
54     }
55 }

```

Рис-11. Models-PlaylistIterator.cs

```

dels > C# Playlist.cs > ...
1  namespace MusicPlayerWeb.Models;
2
    15 references
3  public class Playlist
4  {
    11 references
5      public Guid Id { get; set; } = Guid.NewGuid();
    7 references
5      public string Title { get; set; } = default!;
    9 references
7      public string OwnerUsername { get; set; } = default!;
    1 reference
3      public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
3
    2 references
0      public ICollection<PlaylistTrack> PlaylistTracks { get; set; } = new List<PlaylistTrack>();
1
    1 reference
2      public ITrackIterator CreateIterator(IEnumerable<Track> tracks)
3      {
4          return new PlaylistIterator(tracks);
5      }
5  }
7

```

Рис-12. Models-Playlist.cs

Controllers / PlaylistsController.cs / PlaylistsController / Create

```
1 using Microsoft.AspNetCore.Authorization;
2 using Microsoft.AspNetCore.Mvc;
3 using MusicPlayerWeb.Models;
4 using MusicPlayerWeb.Services;
5 using System.Linq;
6
7 namespace MusicPlayerWeb.Controllers;
8
9 [Authorize]
10 1 reference
11 public class PlaylistsController : Controller
12 {
13     6 references
14     private readonly IPlaylistService _playlists;
15
16     0 references
17     public PlaylistsController(IPlaylistService playlists)
18     {
19         _playlists = playlists;
20     }
21
22     [HttpPost]
23     0 references
24     public IActionResult Create(string title)
25     {
26         var username = User.Identity!.Name!;
27
28         if (string.IsNullOrEmpty(title))
29         {
30             TempData["Error"] = "Назва плейліста є обов'язковою.";
31             return RedirectToAction("Index", "Home");
32         }
33
34         var playlist = _playlists.Create(username, title);
35         return RedirectToAction("Index", "Home", new { playlistId = playlist.Id });
36     }
37
38     [HttpPost]
39     0 references
40     public IActionResult Rename(Guid playlistId, string newTitle)
41     {
42         var username = User.Identity!.Name!;
43
44         if (string.IsNullOrEmpty(newTitle))
45         {
46             TempData["Error"] = "Нова назва не може бути порожньою.";
47             return RedirectToAction("Index", "Home", new { playlistId });
48         }
49
50         var ok = _playlists.Rename(username, playlistId, newTitle);
51
52         if (!ok)
53             TempData["Error"] = "Плейліст не знайдено.";
54
55         return RedirectToAction("Index", "Home", new { playlistId });
56     }
57 }
```

```

[HttpPost]
0 references
public IActionResult Delete(Guid playlistId)
{
    var username = User.Identity!.Name!;

    var ok = _playlists.Delete(username, playlistId);

    if (!ok)
    {
        TempData["Error"] = "Плейліст не знайдено.";
        return RedirectToAction("Index", "Home");
    }

    return RedirectToAction("Index", "Home");
}

[HttpGet]
0 references
public IActionResult View(Guid id)
{
    var username = User.Identity!.Name!;
    var playlist = _playlists.GetById(username, id);

    if (playlist == null)
    {
        TempData["Error"] = "Плейліст не знайдено.";
        return RedirectToAction("Index", "Home");
    }

    var tracks = _playlists.GetTracks(username, id).ToList();

    ViewBag.Playlist = playlist;

    if (!tracks.Any())
    {
        ViewBag.HasIterator = false;
        return View(tracks);
    }

    // Використовуємо патерн Iterator
    var iterator = playlist.CreateIterator(tracks);

    ViewBag.HasIterator = true;
    ViewBag.FirstTrackTitle = iterator.Current().Title;
    ViewBag.TrackTitlesJson = System.Text.Json.JsonSerializer.Serialize(
        tracks.Select(t => t.Title).ToList()
    );

    return View(tracks);
}
}

```

Рис-13-14. Controlllers – PlaylistController.cs

views > Playlists > view.cshtml

```
~/MusicPlayerWeb/Views - Contains emphasized items -Track>
2
3  @{
4      var playlist = (MusicPlayerWeb.Models.Playlist)ViewBag.Playlist;
5      bool hasIterator = ViewBag.HasIterator is bool b && b;
6  }
7
8  <h2>Плейліст: @playlist.Title</h2>
9
10 @if (!Model.Any())
11 {
12     <p>У цьому плейлісті ще немає треків.</p>
13 }
14 else
15 {
16     <h3>Треки</h3>
17     <ul>
18         @foreach (var t in Model)
19         {
20             <li>@t.Title</li>
21         }
22     </ul>
23
24     @if (hasIterator)
25     {
26         <hr />
27         <h3>Ітератор (Iterator) для цього плейліста</h3>
28
29         <p>
30             Поточний трек:
31             <strong id="currentTrack">@ViewBag.FirstTrackTitle</strong>
32         </p>
33
34         <button id="btnPrev">Попередній</button>
35         <button id="btnNext">Наступний</button>
36
37         <script>
38             (function () {
39                 const titles = @Html.Raw(ViewBag.TrackTitlesJson ?? "[]");
40                 let index = 0;
41
42                 const currentEl = document.getElementById("currentTrack");
43                 const prevBtn = document.getElementById("btnPrev");
44                 const nextBtn = document.getElementById("btnNext");
45
46                 function update() {
47                     if (!titles.length) return;
48                     currentEl.textContent = titles[index];
49                 }
50
51                 prevBtn.addEventListener("click", function () {
52                     if (index > 0) {
53                         index--;
54                         update();
55                     }
56                 });
57
58                 nextBtn.addEventListener("click", function () {
59                     if (index < titles.length - 1) {
60                         index++;
61                         update();
62                     }
63                 });
64
65                 update();
66             })();
67         </script>
68     }
69 }
70
```

Рис-15-16. Views-Playlists-View.cshtml

Реалізувати не менше 3-х класів відповідно до обраної теми.

```
dels > Playlist.cs > Playlist > CreateIterator
1 namespace MusicPlayerWeb.Models;
2
3     15 references
4 public class Playlist
5 {
6     11 references
7     public Guid Id { get; set; } = Guid.NewGuid();
8     7 references
9     public string Title { get; set; } = default!;
10    9 references
11    public string OwnerUsername { get; set; } = default!;
12    1 reference
13    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
14
15    2 references
16    public ICollection<PlaylistTrack> PlaylistTracks { get; set; } = new List<PlaylistTrack>();
17
18    1 reference
19    public ITrackIterator CreateIterator(IEnumerable<Track> tracks)
20    {
21        return new PlaylistIterator(tracks);
22    }
23 }
```

Рис-17. Models – Playlist.cs

DbPlaylistService.cs x AppDbContext.cs M HomeController.cs M TracksController.cs M P

Services > DbPlaylistService.cs > DbPlaylistService > AddTrack

```

1  using Microsoft.EntityFrameworkCore;
2  using MusicPlayerWeb.Data;
3  using MusicPlayerWeb.Models;
4
5  namespace MusicPlayerWeb.Services;
6
7  2 references
8  public class DbPlaylistService : IPlaylistService
9  {
10     19 references
11     private readonly AppDbContext _db;
12
13     0 references
14     public DbPlaylistService(AppDbContext db) => _db = db;
15
16     2 references
17     public IEnumerable<Playlist> GetForUser(string username)
18     {
19         return _db.Playlists
20             .Where(p => p.OwnerUsername == username)
21             .OrderBy(p => p.CreatedAt)
22             .ToList();
23     }
24
25     3 references
26     public Playlist? GetById(string username, Guid id)
27     {
28         return _db.Playlists
29             .Include(p => p.PlaylistTracks)
30             .ThenInclude(pt => pt.Track)
31             .FirstOrDefault(p => p.Id == id && p.OwnerUsername == username);
32     }
33
34     2 references
35     public Playlist Create(string username, string title)
36     {
37         var pl = new Playlist
38         {
39             Title = title.Trim(),
40             OwnerUsername = username
41         };
42
43         _db.Playlists.Add(pl);
44         _db.SaveChanges();
45         return pl;
46     }
47
48     3 references
49     public IEnumerable<Track> GetTracks(string username, Guid playlistId)
50     {
51         return _db.PlaylistTracks
52             .Include(pt => pt.Track)
53             .Include(pt => pt.Playlist)
54             .Where(pt => pt.PlaylistId == playlistId &&
55                 pt.Playlist.OwnerUsername == username)
56             .OrderBy(pt => pt.Order)
57             .Select(pt => pt.Track)
58             .ToList();

```

```

32     }
33
34     2 references
35     public bool AddTrack(string username, Guid playlistId, Guid trackId)
36     {
37         var playlist = _db.Playlists.FirstOrDefault(p => p.Id == playlistId && p.OwnerUsername == username);
38         if (playlist == null) return false;
39
40         var exists = _db.PlaylistTracks.Any(pt => pt.PlaylistId == playlistId && pt.TrackId == trackId);
41         if (exists) return true;
42
43         int maxOrder = _db.PlaylistTracks
44             .Where(pt => pt.PlaylistId == playlistId)
45             .Select(pt => (int?)pt.Order)
46             .Max() ?? 0;
47
48         _db.PlaylistTracks.Add(new PlaylistTrack
49         {
50             PlaylistId = playlistId,
51             TrackId = trackId,
52             Order = maxOrder + 1
53         });
54
55         _db.SaveChanges();
56         return true;
57     }
58
59     1 reference
60     public bool RemoveTrack(string username, Guid playlistId, Guid trackId)
61     {
62         var pt = _db.PlaylistTracks
63             .Include(x => x.Playlist)
64             .FirstOrDefault(x =>
65                 x.PlaylistId == playlistId &&
66                 x.TrackId == trackId &&
67                 x.Playlist.OwnerUsername == username);
68
69         if (pt == null) return false;
70
71         _db.PlaylistTracks.Remove(pt);
72         _db.SaveChanges();
73         return true;
74     }
75
76     2 references
77     public bool Rename(string username, Guid playlistId, string newTitle)
78     {
79         var pl = _db.Playlists.FirstOrDefault(p => p.Id == playlistId && p.OwnerUsername == username);
80         if (pl == null) return false;
81
82         pl.Title = newTitle.Trim();
83         _db.SaveChanges();
84         return true;
85     }
86
87     ..

```

Рис-18-19. Services – DbPlaylistService.cs

Controllers > PlaylistsController.cs > PlaylistsController > Create

```
1 using Microsoft.AspNetCore.Authorization;
2 using Microsoft.AspNetCore.Mvc;
3 using MusicPlayerWeb.Models;
4 using MusicPlayerWeb.Services;
5 using System.Linq;
6
7 namespace MusicPlayerWeb.Controllers;
8
9 [Authorize]
10 public class PlaylistsController : Controller
11 {
12     private readonly IPlaylistService _playlists;
13
14     public PlaylistsController(IPlaylistService playlists)
15     {
16         _playlists = playlists;
17     }
18
19     [HttpPost]
20     public IActionResult Create(string title)
21     {
22         var username = User.Identity!.Name!;
23
24         if (string.IsNullOrEmpty(title))
25         {
26             TempData["Error"] = "Назва плейліста є обов'язковою.";
27             return RedirectToAction("Index", "Home");
28         }
29
30         var playlist = _playlists.Create(username, title);
31         return RedirectToAction("Index", "Home", new { playlistId = playlist.Id });
32     }
33
34     [HttpPost]
35     public IActionResult Rename(Guid playlistId, string newTitle)
36     {
37         var username = User.Identity!.Name!;
38
39         if (string.IsNullOrEmpty(newTitle))
40         {
41             TempData["Error"] = "Нова назва не може бути порожньою.";
42             return RedirectToAction("Index", "Home", new { playlistId });
43         }
44
45         var ok = _playlists.Rename(username, playlistId, newTitle);
46
47         if (!ok)
48             TempData["Error"] = "Плейліст не знайдено.";
49
50         return RedirectToAction("Index", "Home", new { playlistId });
51     }
52 }
```

```

53 [HttpPost]
54     0 references
55     public IActionResult Delete(Guid playlistId)
56     {
57         var username = User.Identity!.Name!;
58         var ok = _playlists.Delete(username, playlistId);
59
60         if (!ok)
61         {
62             TempData["Error"] = "Плейліст не знайдено.";
63             return RedirectToAction("Index", "Home");
64         }
65
66         return RedirectToAction("Index", "Home");
67     }
68
69 [HttpGet]
70     0 references
71     public IActionResult View(Guid id)
72     {
73         var username = User.Identity!.Name!;
74         var playlist = _playlists.GetById(username, id);
75
76         if (playlist == null)
77         {
78             TempData["Error"] = "Плейліст не знайдено.";
79             return RedirectToAction("Index", "Home");
80         }
81
82         var tracks = _playlists.GetTracks(username, id).ToList();
83
84         ViewBag.Playlist = playlist;
85
86         if (!tracks.Any())
87         {
88             ViewBag.HasIterator = false;
89             return View(tracks);
90         }
91
92         var iterator = playlist.CreateIterator(tracks);
93
94         ViewBag.HasIterator = true;
95         ViewBag.FirstTrackTitle = iterator.Current().Title;
96         ViewBag.TrackTitlesJson = System.Text.Json.JsonSerializer.Serialize(
97             tracks.Select(t => t.Title).ToList()
98         );
99         return View(tracks);
100     }
101 }

```

Рис-20-21. Controllers – PlaylistsControllers.cs

Висновок:

У ході виконання лабораторної роботи було опрацьовано базові патерни проектування, такі як Singleton, Iterator, Proxy, State та Strategy. Це дозволило сформувати цілісне розуміння того, як шаблони допомагають створювати гнучкі, розширювані та зрозумілі програмні системи. На прикладі веб-застосунку «Музичний програвач» було реалізовано патерн Iterator, який використано для організації послідовного доступу до

треків у плейлісті без необхідності розкривати внутрішню структуру колекції. Ітератор забезпечив зручний механізм навігації між треками та дозволив чітко відокремити логіку зберігання даних від механізму їх перебору. Практичне застосування патерну показало, наскільки важливо правильно організовувати взаємодію між частинами системи, щоб спростити подальший розвиток та підвищити якість коду. Виконання роботи дало змогу закріпити теоретичні знання про шаблони проєктування та на практиці переконатися в їх ефективності при створенні реальних програмних рішень.

Відповіді на контрольні запитання:

1. Що таке шаблон проєктування?

Шаблон проєктування — це типове рішення поширеної задачі проєктування програмного забезпечення, перевірене практикою.

2. Навіщо використовувати шаблони проєктування?

Вони роблять код зрозумілішим, зручнішим для повторного використання, легшим у підтримці та розширенні.

3. Яке призначення шаблону «Стратегія»?

Дозволяє визначати сімейство алгоритмів, інкапсулювати кожен із них та робити їх взаємозамінними під час виконання.

4. Нарисуйте структуру шаблону «Стратегія».

(Тут можна вставити діаграму, як у тебе вище: Context → Strategy → ConcreteStrategy1/2).

5. Які класи входять в шаблон «Стратегія», та яка між ними взаємодія?

- Context – клас, який використовує стратегію.
- Strategy – інтерфейс для всіх стратегій.
- ConcreteStrategy – конкретні реалізації алгоритмів.

Context тримає посилання на Strategy і виконує метод вибраної реалізації.

6. Яке призначення шаблону «Стан»?

Дозволяє об'єкту змінювати свою поведінку залежно від внутрішнього стану, ніби він змінює свій клас.

7. Нарисуйте структуру шаблону «Стан».

(Діаграма: Context → State → ConcreteStateA / ConcreteStateB).

8. Які класи входять в шаблон «Стан», та яка між ними взаємодія?

- Context – клас, який зберігає поточний стан.
- State – інтерфейс станів.
- ConcreteState – реалізації різних станів.

Context делегує виконання поточному стану, який може змінити його на інший.

9. Яке призначення шаблону «Ітератор»?

Дозволяє послідовно перебирати елементи колекції, не розкриваючи її внутрішньої структури.

10. Нарисуйте структуру шаблону «Ітератор».

(Діаграма: Iterator ↔ ConcreteIterator ↔ Aggregate ↔ ConcreteAggregate).

11. Які класи входять в шаблон «Ітератор», та яка між ними взаємодія?

- Iterator – інтерфейс для доступу до елементів.
- ConcreteIterator – конкретна реалізація.

12. В чому полягає ідея шаблону «Одинак»?

Забезпечити існування лише одного екземпляра класу та надати глобальну точку доступу до нього.

13. Чому шаблон «Одинак» вважають «анти-шаблоном»?

Бо він порушує принципи ООП: ускладнює тестування, створює приховані залежності та глобальний стан.

14. Яке призначення шаблону «Проксі»?

Контролює доступ до іншого об'єкта, виступаючи його замісником (наприклад, ліниве завантаження, кешування, контроль доступу).

15. Нарисуйте структуру шаблону «Проксі».

(Діаграма: Client → Proxy → RealSubject ← Interface).

16. Які класи входять в шаблон «Проксі», та яка між ними взаємодія?

- Subject – спільний інтерфейс.
- RealSubject – реальний об'єкт.
- Proxy – замісник, який містить посилання на RealSubject і контролює звернення клієнта.

